

Advanced Educational Content Creation System

Project Presentation — Full Pipeline Overview

Talk Structure (30 min)

#	Section	Time
1	Motivation & Problem	3 min
2	Stage 1: Concept Graph Pipeline	10 min
3	Live Demo	5 min
4	Stage 2: State Machine Architecture	8 min
5	Full Vision & Next Steps	4 min

1. Motivation (3 min)

The problem: Textbooks are static. They encode expert knowledge linearly with no awareness of what a reader already knows, where they're struggling, or how they learn best.

The goal: Build a system that transforms any textbook into an adaptive, interactive learning experience — automatically, at scale.

Two-stage pipeline:

Stage 1 (COMPLETE)		Stage 2 (ARCHITECTURE)
Raw textbook PDF / MD / QMD	→	Concept Graph 1,267 nodes ~2,400 edges
		→ Adaptive Tutor State Machine Student Model

Scope: Initially built on *Foundations of Computer Vision* (55 chapters). Next: 200–400 STEM textbooks.

2. Stage 1: Concept Graph Pipeline (10 min)

2.1 What the Pipeline Does

Takes a raw textbook in any format and produces a richly annotated knowledge graph.

Input formats supported: - .qmd (Quarto Markdown — the target format for the vision textbook)
- .md (standard Markdown) - .pdf (via ingestion stage)

Three types of extracted elements:

Element	What it is	Example
Concepts	Atomic knowledge units	phong_reflection_model, backpropagation
Edges	Prerequisite relationships	lambertian_surface → phong_reflection_model
Items	Linked content	Exercises, figures, theorems, examples per concept

2.2 Concept Node — What Each Node Contains

Every concept node is a structured record:

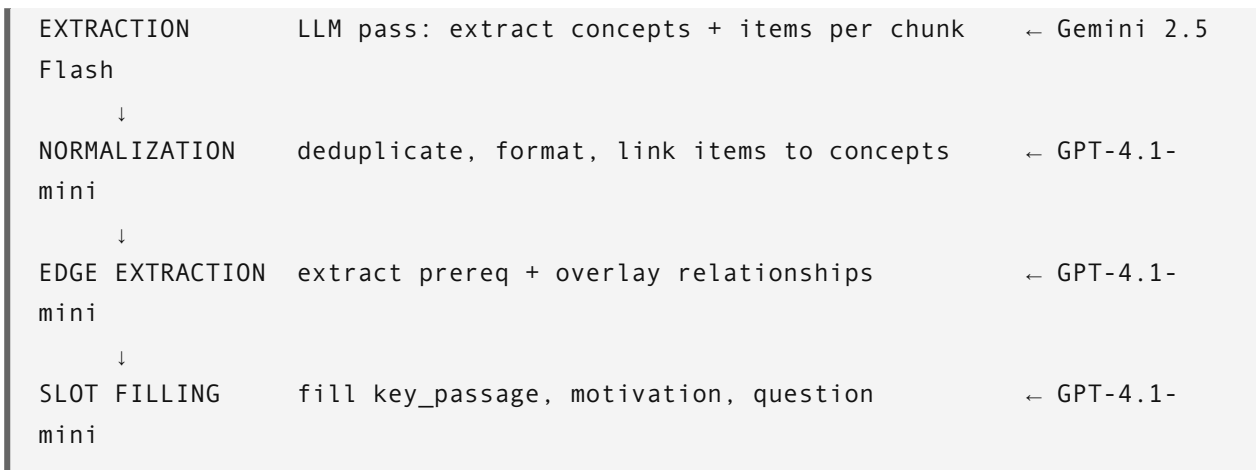
```
id:           phong_reflection_model
kind:         definition | theorem | technique | idea
title:        "Phong Reflection Model"

content:      paraphrased book text
key_passage:  verbatim sentence from the book
motivation:   why this concept matters
question:     study question
recap_md:     bullet summary

item_ids:     → linked figures, exercises, examples
position:     chapter 5, section 5.2, book_order 214
prereqs:      [lambertian_surface, surface_albedo]
```

2.3 The 21-Stage Pipeline

```
INGESTION      parse structure, download images, enrich text
  ↓
CHUNKING       segment into ~2000-token chunks
  ↓
```



Current results on Foundations of Computer Vision:

Metric	Value
Total concepts extracted	1,267
Chapters covered	55
Prerequisite edges	~2,400
Concepts with verbatim key passage	98%
Concepts with study question	10% (book-grounded only)
Linked figures	871

2.4 Edge Types

Edges encode two types of relationships:

- **Prerequisite** (requires, special_case_of, generalizes, formalizes) — concept A must be understood before concept B
- **Overlay** (illustrates, used_to_prove, see_also, contrast_with) — conceptual connections without strict ordering

Each edge carries: from, to, kind, strength, rationale (evidence sentence from the book).

2.5 Challenges

Where extraction works well: - Dense mathematical chapters (Ch. 15, 16, 32) — every named term is a clearly bounded concept - Chapters with explicit definitions, theorems, proofs

Where extraction is harder: - Qualitative/applied chapters (Ch. 7, 9) — sub-ideas are implicit, not explicitly named - CS textbooks — concepts are often described narratively without clear delimiters - Contrast with math textbooks, where terminology and PSets provide natural anchors

Comparison with existing pipelines (Metacademy, DeepTutor): - Metacademy: hand-curated, does not scale - DeepTutor: LLM-based extraction, but focuses on dialogue not graph structure - This pipeline: automated, full book, richly annotated, prerequisite-grounded

3. Live Demo (5 min)

Three visualizations, all running locally:

Demo 1 — Full Book Graph (`concept-graph-vision.html`) - 1,267 nodes clustered by chapter - Nodes positioned within each cluster in DAG order (prereqs left → dependents right) - Cross-chapter edges visible at zoom out - Click any node → panel shows: definition, key passage, motivation, prerequisites, figures, exercises

Demo 2 — Chapter View (`concept-graph.html`) - Per-chapter DAG with Dagre layout - Nodes colored by kind (definition / theorem / technique / idea) - Arrows show prerequisite direction - Click node → full concept panel with all slot data

Demo 3 — Concept Panel Walk through a specific concept (e.g. Phong Reflection Model): - Verbatim key passage from the book - Motivation sentence - Linked figure - Linked exercise - Prerequisite concepts

4. Stage 2: State Machine Architecture (8 min)

4.1 The Core Idea

The concept graph node contains everything needed to generate a lesson. The state machine compiles it and executes it.

```
Concept Node → Lesson Plan (compiled once) → State Machine (runs at session time)
```

4.2 Inspiration from Game Architecture

Game engines solve a similar problem: large, structured worlds that a player navigates non-linearly, with persistent state and adaptive difficulty.

Three patterns directly adopted:

Hierarchical State Machine (HSM) States can contain sub-states. A student in Chapter 5 / Concept 3 / Practice sub-state — exiting cleanly from any depth always saves progress.

Behavior Tree (Socratic Decision Logic) In games, AI behavior trees decide what an enemy does without the enemy knowing how to render itself. Here: the behavior tree decides what the tutor does without the LLM choosing the strategy.

Code decides: when to give a hint, when to ask an MCQ, when to advance
 LLM generates: the actual sentence

Lookup Table + Timer (Spaced Repetition) Game loops use frame-indexed lookup tables. The session scheduler uses a cooldown timer per concept — exactly like ability cooldowns in games — to implement spaced repetition.

4.3 The Lesson State Machine

Each concept compiles to a fixed lesson sequence:

```

HOOK          ← concept.question
MOTIVATE      ← concept.motivation.text
EXPLAIN       ← concept.content
VISUAL        ← concept.item_ids.figures
KEY MOMENT    ← concept.key_passage
EXAMPLE       ← concept.item_ids.examples
PRACTICE      ← concept.item_ids.exercises
  └─ ATTEMPT
  └─ HINT (after 3 wrong)
  └─ RETRY
RECAP         ← concept.recap_md
  
```

The same lesson plan drives three output formats: interactive book reading, slide deck, video script.

4.4 The Behavior Tree Replaces Prompt Rules

The current active reader tutor has Socratic rules written in English inside a prompt. The LLM interprets them — and can misinterpret them.

```

CURRENT:  "If clearly stuck, give ONE short hint" (LLM decides what 'stuck'
means)
PROPOSED:  if attempts >= 3 → call LLM('give_hint', concept.key_passage)
  
```

The behavior tree makes decisions in code. The LLM only generates text for the decision that was already made.

4.5 The Student Model

A persistent record that survives sessions:

```
per concept:  status, attempts, retention_score, next_due
struggle profile: weak concept tags, stuck concepts
interaction log: questions asked, hints taken, time spent
```

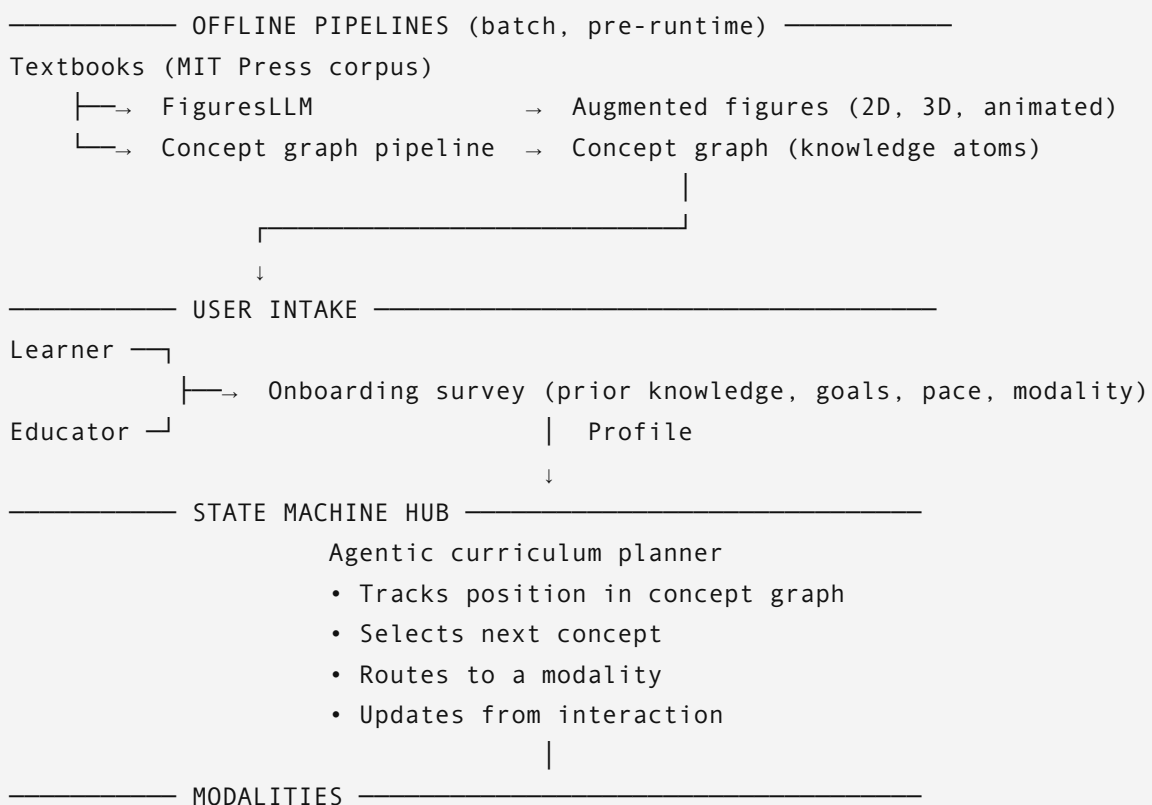
The session scheduler reads `next_due` per concept to decide what to review — implementing spaced repetition as a cooldown system.

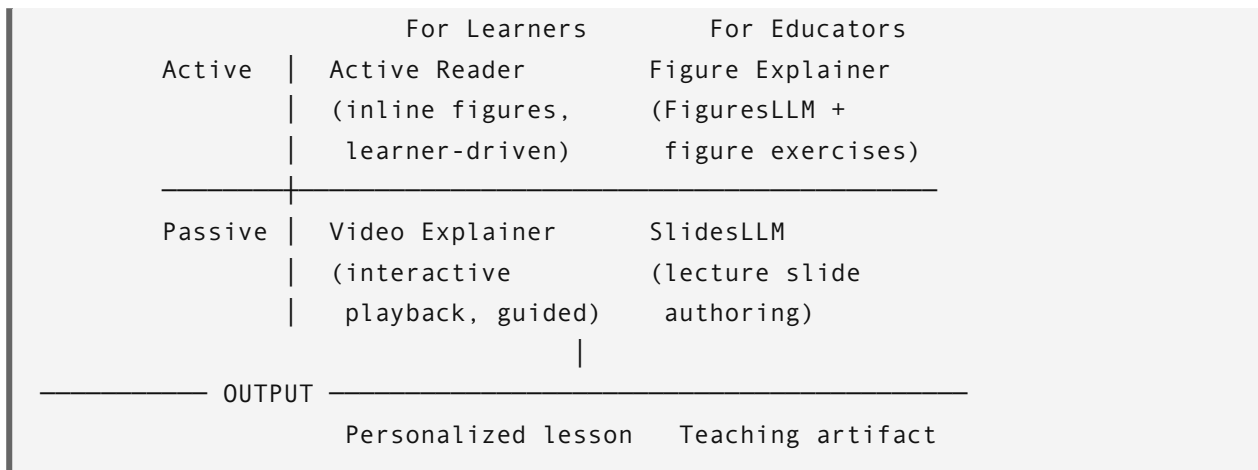
4.6 Multiple State Machine Levels

Session Scheduler	→ which concept to study next (spaced repetition)
Book/Chapter SM	→ LOCKED / AVAILABLE / IN_PROGRESS / COMPLETE
Lesson SM	→ HOOK → EXPLAIN → PRACTICE → RECAP
Behavior Tree	→ hint? MCQ? praise? Socratic question?
Student Model	→ persistent knowledge state

5. Full Vision & Next Steps (4 min)

5.1 The Complete System Map





5.2 What's Built vs. Planned

Component	Status
Concept graph pipeline (21 stages)	COMPLETE
Concept graph visualization (3 views)	COMPLETE
Active Reader /api/chat	RUNNING
FiguresLLM pipeline	In progress
Onboarding survey	Planned
State machine hub	Architecture designed
Video Explainer	Planned
SlidesLLM	Planned

5.3 Build Order (Immediate Next Steps)

Priority	Task
1	Lesson plan compiler — concept node → lesson_plan.json per concept
2	Lesson state machine implementation (XState or custom)
3	Student model persistence layer
4	Behavior tree replacing prompt rules in /api/chat
5	Session scheduler (spaced repetition)
6	Slides + video script renderers

5.4 Scale Plan

- Current: 1 textbook, 55 chapters, 1,267 concepts
- Next: 200–400 STEM textbooks through the same pipeline
- Math textbooks first (highest extraction quality)
- CS textbooks with improved sub-idea extraction

Summary

STAGE 1 (BUILT)	STAGE 2 (PLANNED)
Raw textbook	Concept graph
↓	↓
21-stage LLM pipeline	Lesson plan compiler
↓	↓
1,267 concept nodes	Hierarchical state machine
+ 2,400 edges	↓
+ 871 figures	Behavior tree (Socratic logic)
+ exercises/items	↓
↓	Student model (persistent)
Interactive visualizations	↓
(running today)	3 output formats
	(book / slides / video)

The concept graph is the bridge. Everything in Stage 2 is compiled from it.